# Alexandre Baron's portfolio

—

# Selected computer graphics works

The following pages showcase personal projects and hobby work in computer graphics and rendering, each accompanied by a brief description of the underlying technology.

All samples were rendered in a personal 3D game engine, unless noted otherwise.

Not all projects are open source, but source code is available upon request.

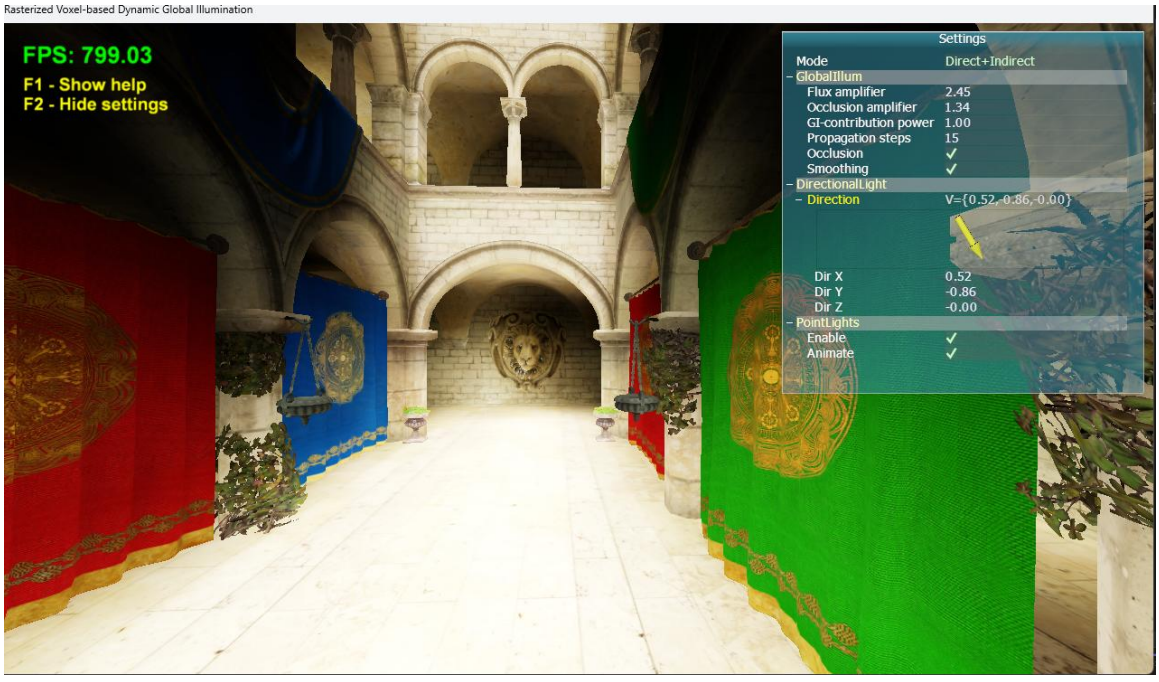
The rest of my portfolio is available at:
https://scylardor.github.io/portfolio/

# Voxel Cone Tracing Dynamic Global Illumination

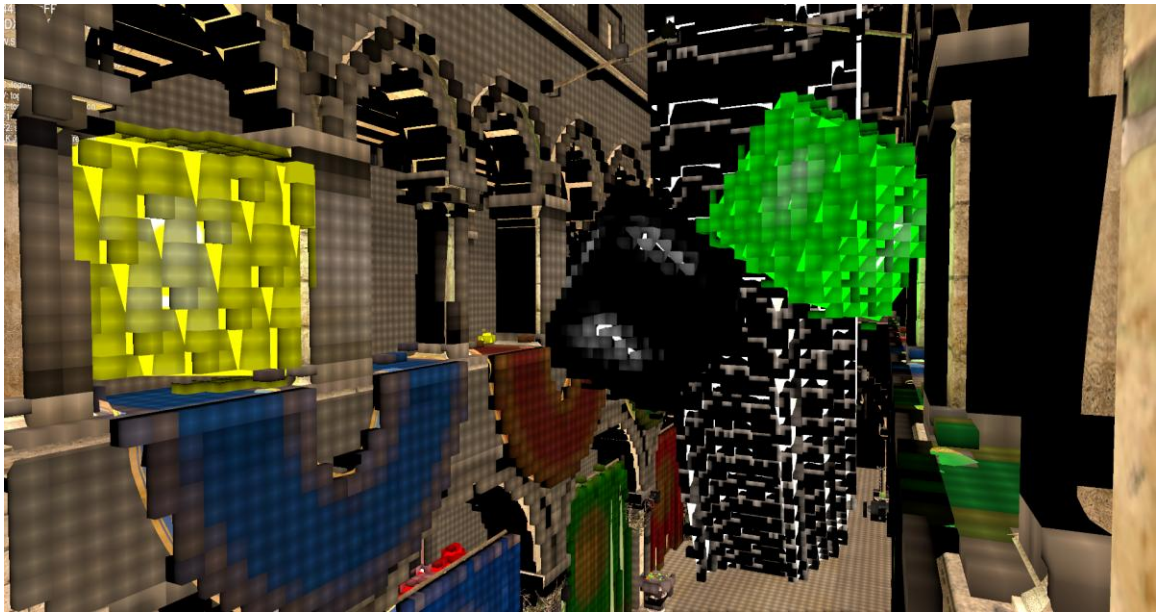
Voxel Cone Tracing avoids the high computational cost of full ray tracing while providing better results than simple, local ambient illumination.

This technique approximates indirect lighting by voxelizing a 3D scene and tracing cones through it.

This implementation builds on an existing codebase, modernized with SDL3 to run on a current Windows 11 machine.

The debug display uses a geometry shader to display voxels.



## Physically based materials

A showcase of materials rendered using the classic metallic/roughness PBR workflow, targeting a photorealistic look. Featured materials include brushed metal (with rust stains), gold, grass, plastic, and brick.

This shader uses a combination of normal mapping, ambient occlusion, and image-based lighting.

It samples an HDR environment cube map to compute diffuse irradiance, and uses a Cook-Torrance BRDF to compute the specular part with:

- Trowbridge-Reitz (GGX) as normal distribution function (accounts for the fraction of facets which reflect light at the viewer)
- Smith's method for Schlick-GGX as a geometric attenuation function (accounts for the shadowing of facets by one another)
- Fresnel-Schlick approximation as a Fresnel function (makes rays with high angle of incidence reflect with higher specular power)

Finally, tone mapping and gamma correction are applied.

Shader code excerpt (fragment shader):

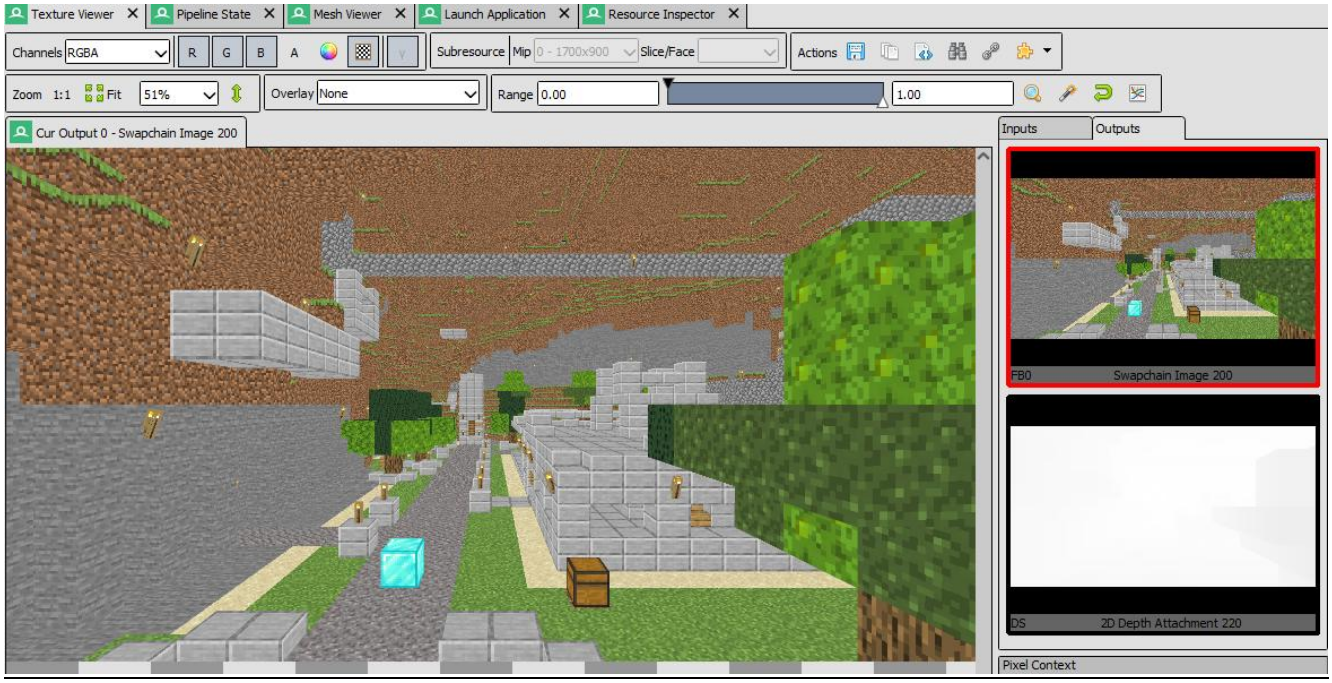
```
void main()
{
    PBR.albedo = texture(albedoMap, vs_texCoords).rgb;
    PBR.metallic = texture(metallicMap, vs_texCoords).r;
    PBR.roughness = texture(roughnessMap, vs_texCoords).r;
    PBR.ao = texture(aoMap, vs_texCoords).r;

    vec3 FragNormalWorld = getNormalFromMap();
    vec3 ViewDirWorld = normalize(Camera.cameraPos.xyz - vs_fragPosWorld);

    // calculate reflectance at normal incidence for Fresnel Schlick function;
    // Use F0 = 0.04 if dia-electric (like plastic) or use the albedo color as F0 if it's a metal / conductor (metallic workflow)
    vec3 F0 = mix(vec3(0.04), PBR.albedo, PBR.metallic);

    // lighting using reflectance equation
    vec3 Lo = vec3(0.0);

    for (int iLight = 0; iLight < Lights.lightsNumber; iLight++)
    {
        // calculate per-light radiance
        vec3 L = normalize(Lights.lightsData[iLight].lightPosition.xyz - vs_fragPosWorld);
        vec3 H = normalize(ViewDirWorld + L);
        float distance = length(Lights.lightsData[iLight].lightPosition.xyz - vs_fragPosWorld);
        float attenuation = 1.0 / (distance * distance);
        vec3 radiance = Lights.lightsData[iLight].lightDiffuse.xyz * attenuation;

        // Cook-Torrance BRDF
        float NDF = TRGGX_NDF(FragNormalWorld, H, PBR.roughness);
        float G = SmithSchlickGGX_GF(FragNormalWorld, ViewDirWorld, L, PBR.roughness);
        vec3 F = FresnelSchlick_FE(clamp(dot(H, ViewDirWorld), 0.0, 1.0), F0);

        vec3 nominator = NDF * G * F;
        float denominator = 4 * max(dot(FragNormalWorld, ViewDirWorld), 0.0) * max(dot(FragNormalWorld, L), 0.0);
        vec3 specular = nominator / max(denominator, 0.001); // prevent divide by zero for NdotV=0.0 or NdotL=0.0
```

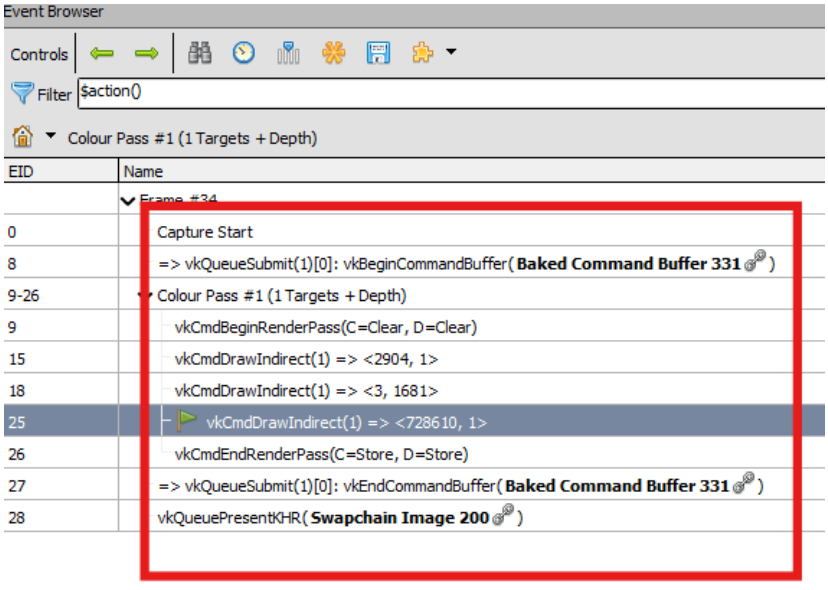
```
        // multiply kD by the inverse metalness such that only non-metals
        // have diffuse lighting, or a linear blend if partly metal (pure metals
        // have no diffuse light).
        kD *= 1.0 - PBR.metallic;

        // scale light by NdotL
        float NdotL = max(dot(FragNormalWorld, L), 0.0);

        // add to outgoing radiance Lo
        Lo += (kD * PBR.albedo / M_PI + specular) * radiance * NdotL;  // note that we already multiplied the BRDF by the Fresnel
    }

    // ambient lighting (note that IBL will replace this ambient lighting with environment lighting).
    vec3 ambient = vec3(0.03) * PBR.albedo * PBR.ao;

    vec3 color = ambient + Lo;


    // HDR tonemapping
    if (ToneMapping.enabled)
    {
        if (ToneMapping.useReinhard)
        {
            // Reinhard tone mapping divides the entire HDR color values to LDR color values.
            // Evenly balances out all brightness values onto LDR, but it does tend to slightly favor brighter areas.
            color /= (color + vec3(1.0));
        }
        else // use exposure
        {
            color = vec3(1.0) - exp(-color * ToneMapping.exposure);
        }
    }

    // gamma correct
    color = pow(color, vec3(1.0/2.2));

    FragColor = vec4(color, 1.0);
```

# Indirect Drawing (Vulkan)

Based on a tutorial, this project renders a Minecraft-style world in as few draw calls as possible. Indirect drawing is a GPU-driven rendering technique where draw parameters are stored in GPU memory rather than set by the CPU, reducing CPU-GPU synchronization overhead and enabling significantly higher performance.

Captures in Renderdoc:



## Game Engine Fundamentals

Outside of professional work, teaching is a passion — which is why one of my personal projects is an online course covering the fundamental mathematics needed to work in video games: trigonometry, linear algebra, vectors, matrices, and quaternions.

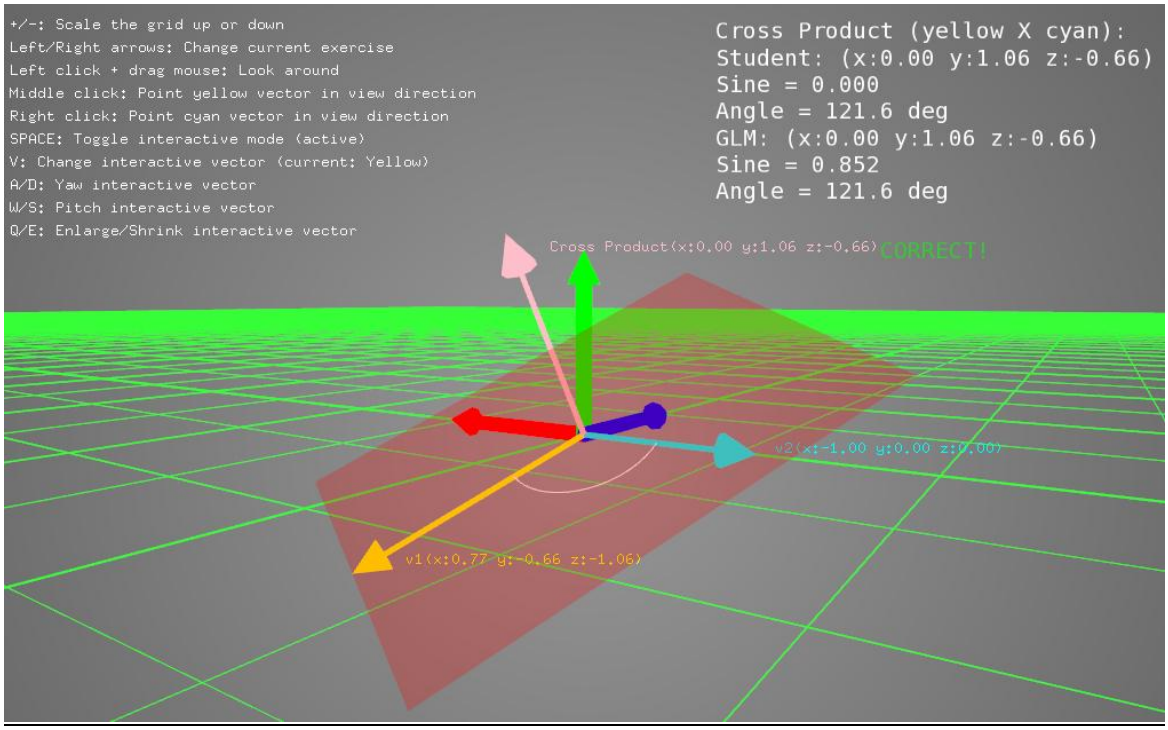
Students learn by implementing a mathematics library in C++ from scratch.

Example programs will be provided for students to validate their implementation against a reference built with the GLM library.

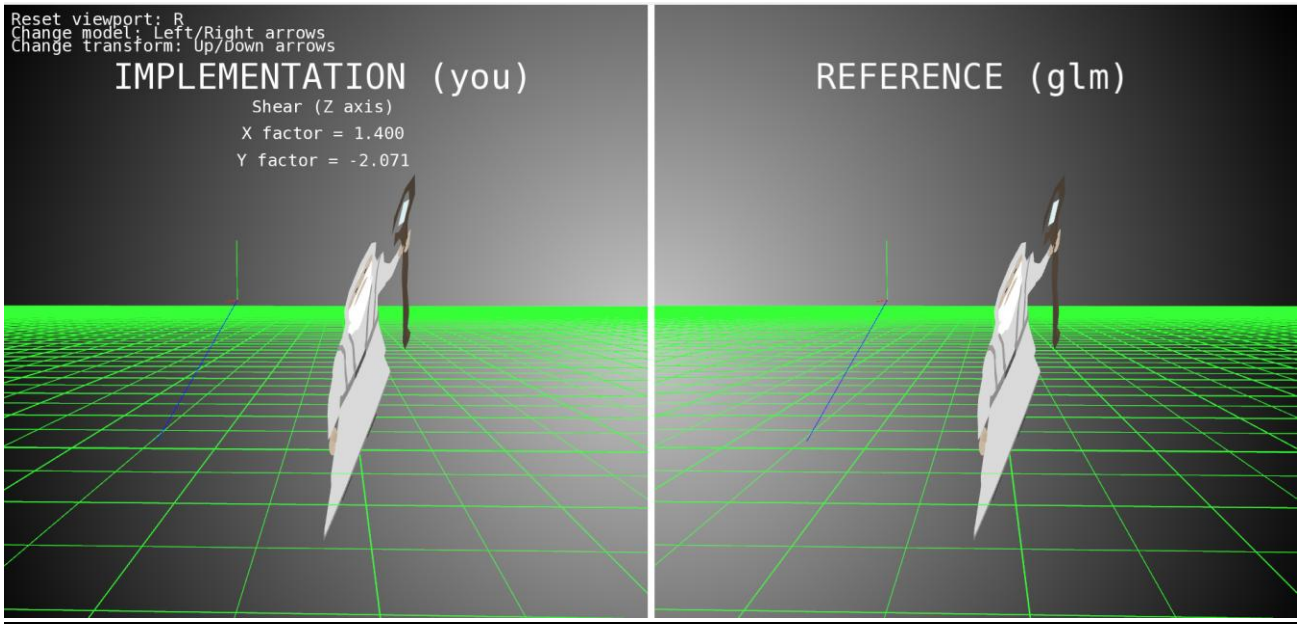
Made in C++, with the OpenFrameworks creative coding toolkit.

The full mathematics library and three exercise programs are almost complete. The course is not yet published online.

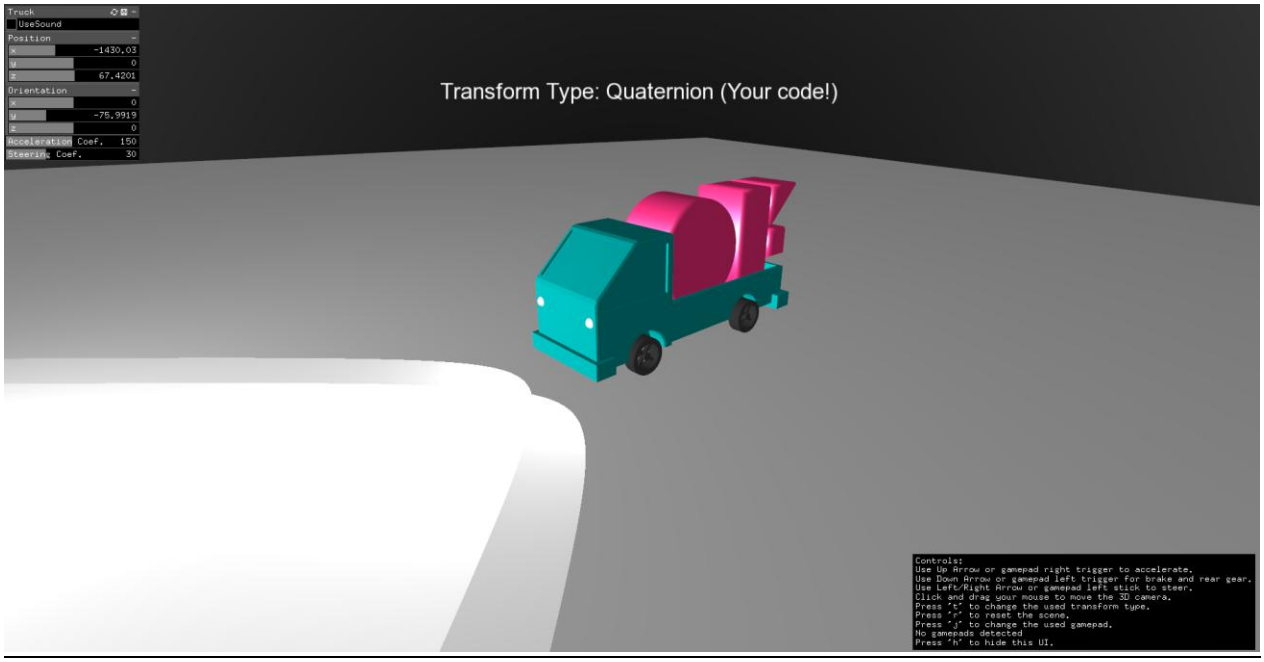
Program 1: Vector Math playground



Program 2: 3D Transforms comparisons (left = student, right = reference)




Program 3: Programming a truck driving simulation (a node-based hierarchy)



## Front Line Zero

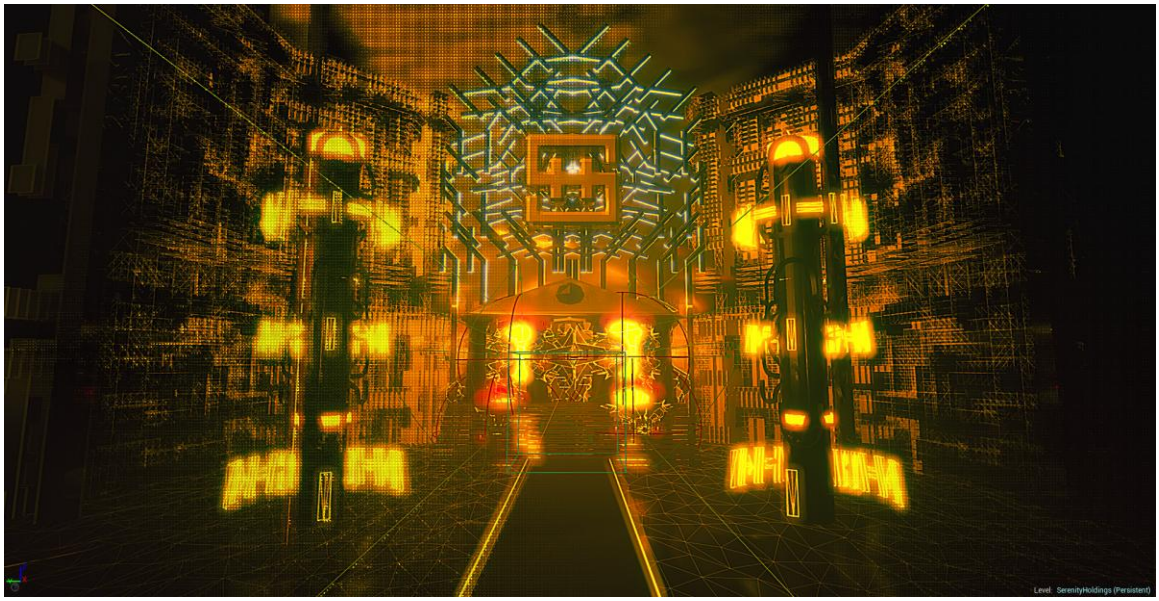
Front Line Zero was an Unreal Engine 4.17 game project I contributed to from 2017 to 2021, led by technical artist Daniel Carter.

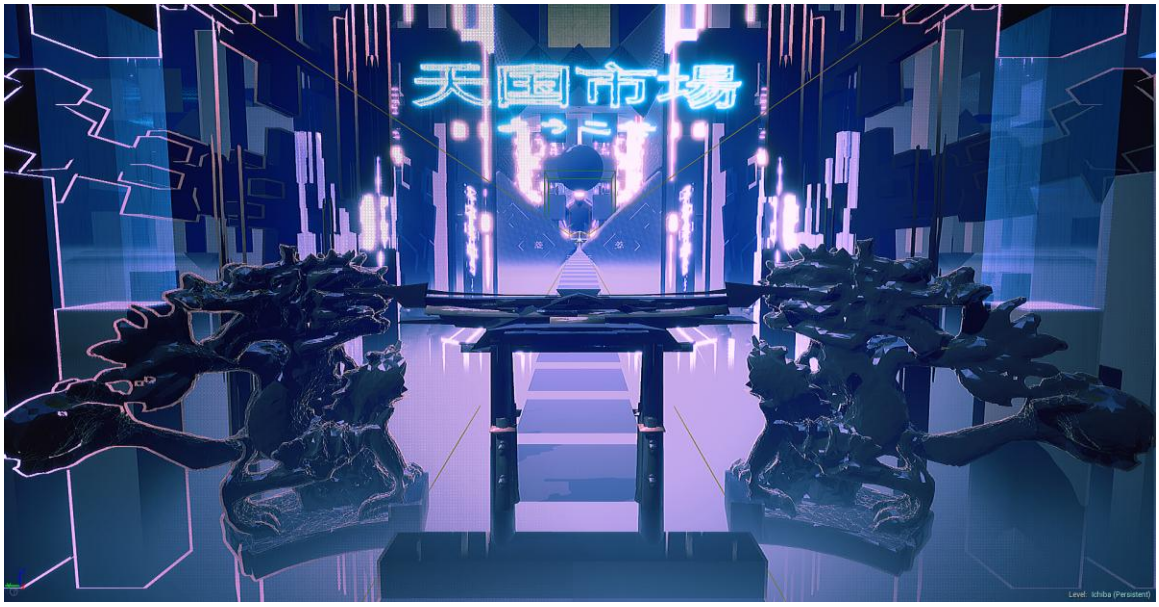
The game was conceived as a psychedelic audio-reactive experience — a 3D rail-shooter in the vein of REZ or Child of Eden, set in a cyberpunk future where a futuristic cyberspace is distorted by psychoactive substances.

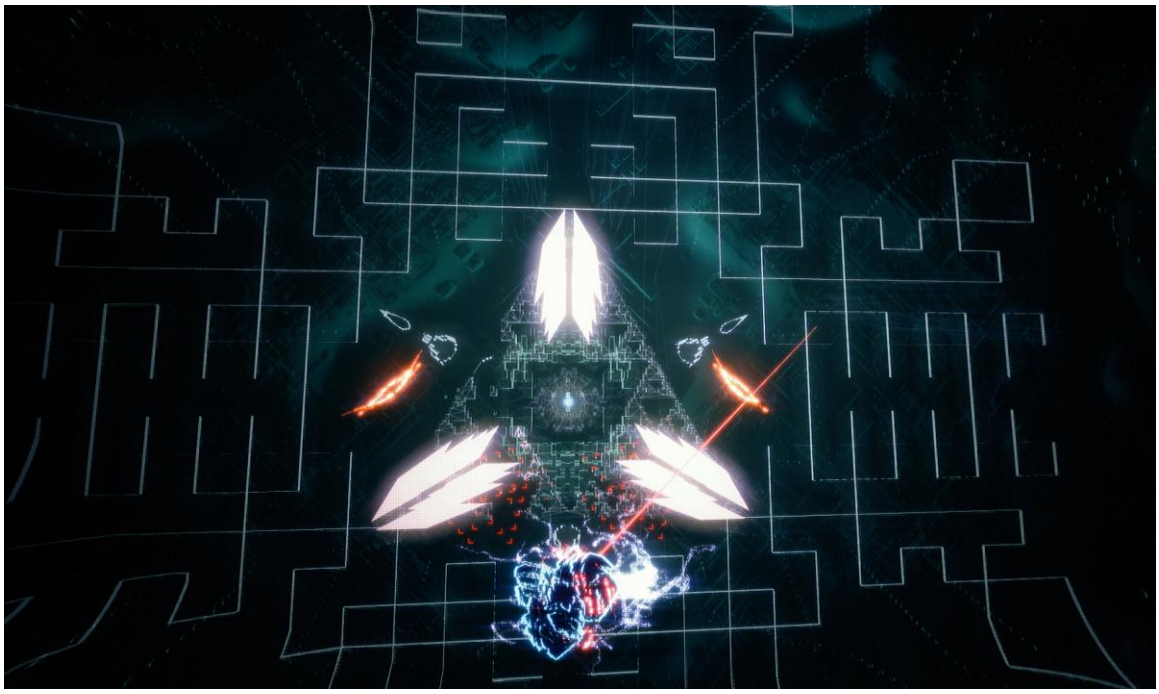
The project had a distinctive art style: most environments were built with procedural generation, and characters including the player's avatar were driven by geometry caches imported from Houdini via the Alembic format rather than a traditional skeletal animation pipeline.

We used the Chameleon plugin for post-processing effects, PopcornFX for particle simulations, and MIDI Asset to read music data in real-time.

Development stopped in 2021, but I still have all the source code and assets.




天国市場



## Unreal Engine Mastering Materials

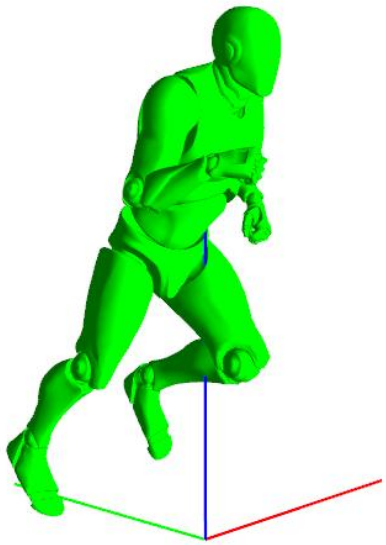
I wanted to know more about how Unreal Engine handles materials and how to work with them from a programmer's perspective, so I took the course "Materials Master Learning" by Sjoerd de Jong to improve my knowledge about the rendering tools and performance viwers available in Unreal Engine.


Shader complexity viewer — Unreal Engine 5

## Animated Mesh Skinning

A teaching demo demonstrating GPU-based hardware skinning. Supports animation crossfading and multiple blending strategies (Lerp, NLerp).

Made in C++ with OpenGL and GLSL shaders.

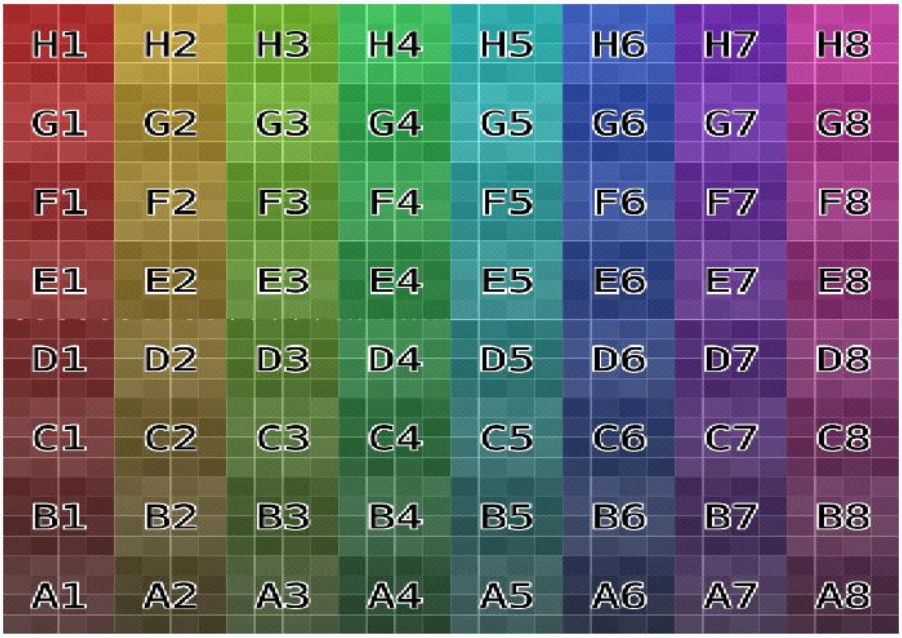


```
uniform mat4 modelViewMatrix;

uniform SkinningMatrices
{
    uniform mat4 mat[64];
} skin;

// Vertex Shader
////////////////////////////////////////////////////////////////////////////////
void main(void)
{
    vec4 defaultPos = vec4(inputPosition, 1.0f);
    vec4 animationPos = vec4(0, 0, 0, 1);
    vec3 defaultNormal = normal;
    vec3 animationNormal = vec3(0, 0, 0);
    for (int i = 0; i < 4; ++i)
    {
        animationPos += (skin.mat[int(boneIndices[i])] * defaultPos) * boneWeights[i];
        animationNormal += (mat3(skin.mat[int(boneIndices[i])]) * defaultNormal) * boneWeights[i];
    }
    gl_Position = sm.projectionMatrix * (modelViewMatrix * vec4(animationPos.xyz, 1.0f));
    outNormal = normalize(mat3(modelViewMatrix) * animationNormal);
}
```

## Software Rasterizer

A complete CPU-side graphics pipeline implemented in C++ with SDL2, without relying on any rendering API.

Features polygon clipping, perspective projection, perspective-correct texture mapping, and vertex attribute interpolation using barycentric coordinates.



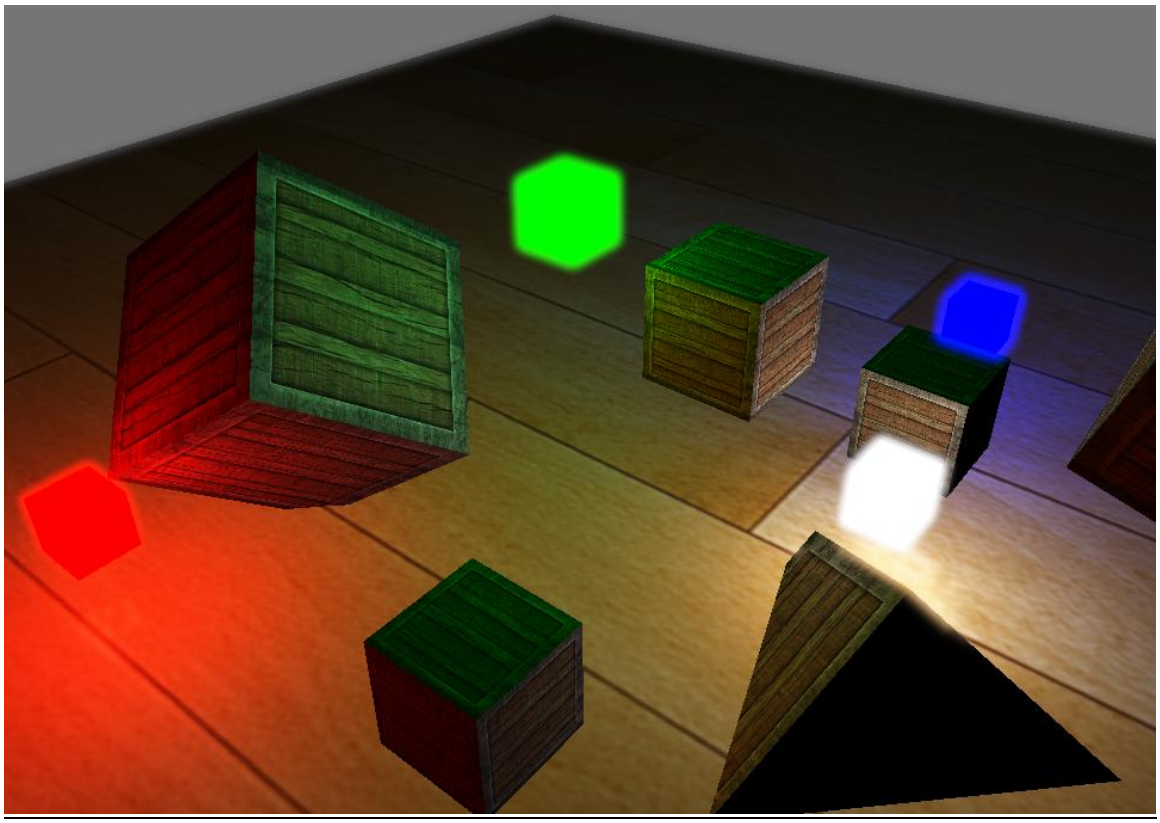

---

```cpp
// Juan Pineda algorithm.
// We do substractions of 2D cross products between the (vtx0 -> vtx1) vector and each of vectors (vtx0 -> pt) and (vtx1 -> pt).
// If the result is positive, it means we're on the "left" side of the edge.
// So a point is inside the triangle if this point is on the left side of all the edges.
int TrianglePrim::EdgeFunction(int px0, int py0, int px1, int py1, int px2, int py2)
{
    // this could be sped up using integers I guess ? and it doesn't do any form of overflow check.
    // see https://fgiesen.wordpress.com/2013/02/08/triangle-rasterization-in-practice/ for ideas to improve.
    int edgeTestResult = (px2 - px0) * (py1 - py0) - (py2 - py0) * (px1 - px0);
    return edgeTestResult;
}


float TrianglePrim::FEdgeFunction(float px0, float py0, float px1, float py1, float px2, float py2)
{
    float edgeTestResult = (px2 - px0) * (py1 - py0) - (py2 - py0) * (px1 - px0);
    return edgeTestResult;
}


void TrianglePrim::ComputeBarycentricSteps()
{
    // Compute base barycentric coordinates for current triangle
    // we apply the top-left bias to make sure that when two non-overlapping triangles share an edge,
    // if you render both of them, each pixel gets processed only once.
    W0Row = EdgeFunction(X1, Y1, X2, Y2, Bounds.MinX(), Bounds.MinY()) + Bias0;
    W1Row = EdgeFunction(X2, Y2, X0, Y0, Bounds.MinX(), Bounds.MinY()) + Bias1;
    W2Row = EdgeFunction(X0, Y0, X1, Y1, Bounds.MinX(), Bounds.MinY()) + Bias2;
```

## Bloom

A straightforward implementation of the Bloom post-processing effect, without attempting physical accuracy.

To implement Bloom, we extract a lit scene's HDR color buffer and an image with only its bright regions visible using a threshold. This extracted brightness image is then blurred and the result added on top of the original HDR scene texture. The bright regions of the scene appear to glow or *bleed* light because they are extended in both width and height by the blur filter. This uses Gaussian blur and ping-pong framebuffers to render the effect.



## Shader code excerpt (fragment):

```
layout (std140, binding = 11) uniform TwoPassGaussianBlurParams
{
    bool        horizontal; // is this the horizontal pass or the vertical pass ?
    // Dimensions of the texture to sample
    int         textureWidth;
    int         textureHeight;
    uint        weightsSize;    // number of weights to read in the buffer
    float       weightsBuffer[TWO_PASS_GAUSSIAN_BLUR_MAX_WEIGHTS];
} GaussBlurParams;


layout(binding = 0) uniform sampler2D image;



void main()
{
    vec2 tex_offset = 1.0 / ivec2(GaussBlurParams.textureWidth, GaussBlurParams.textureHeight); // gets size of single texel
    vec3 result = texture(image, vs_texCoords).rgb * GaussBlurParams.weightsBuffer[0]; // current fragment's contribution

    if (GaussBlurParams.horizontal)
    {
        for (int iWeight = 1; iWeight < GaussBlurParams.weightsSize; ++iWeight)
        {
            result += texture(image, vs_texCoords + vec2(tex_offset.x * iWeight, 0.0)).rgb * GaussBlurParams.weightsBuffer[iWeight];
            result += texture(image, vs_texCoords - vec2(tex_offset.x * iWeight, 0.0)).rgb * GaussBlurParams.weightsBuffer[iWeight];
        }
    }
    else
    {
        for (int iWeight = 1; iWeight < GaussBlurParams.weightsSize; ++iWeight)
        {
            result += texture(image, vs_texCoords + vec2(0.0, tex_offset.y * iWeight)).rgb * GaussBlurParams.weightsBuffer[iWeight];
            result += texture(image, vs_texCoords - vec2(0.0, tex_offset.y * iWeight)).rgb * GaussBlurParams.weightsBuffer[iWeight];
        }
    }
```

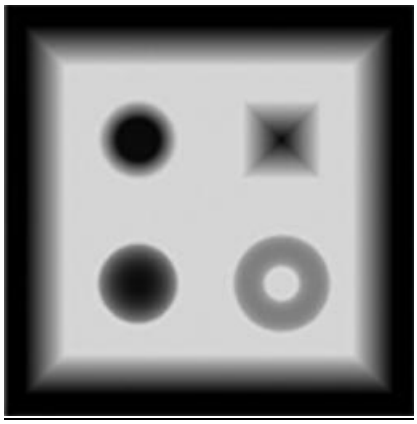
## Parallax Occlusion Mapping

An implementation of Parallax Occlusion Mapping, a technique that creates the illusion of depth on a flat surface.

Notably, it requires no extra vertex data to convey depth. Combined with normal mapping, texture coordinates are offset based on view direction and a heightmap so that each fragment appears higher or lower than its actual position on the surface.

The offset texture coordinates are then used to sample the normal map.

## Used height map:



## Shader code excerpt (fragment):

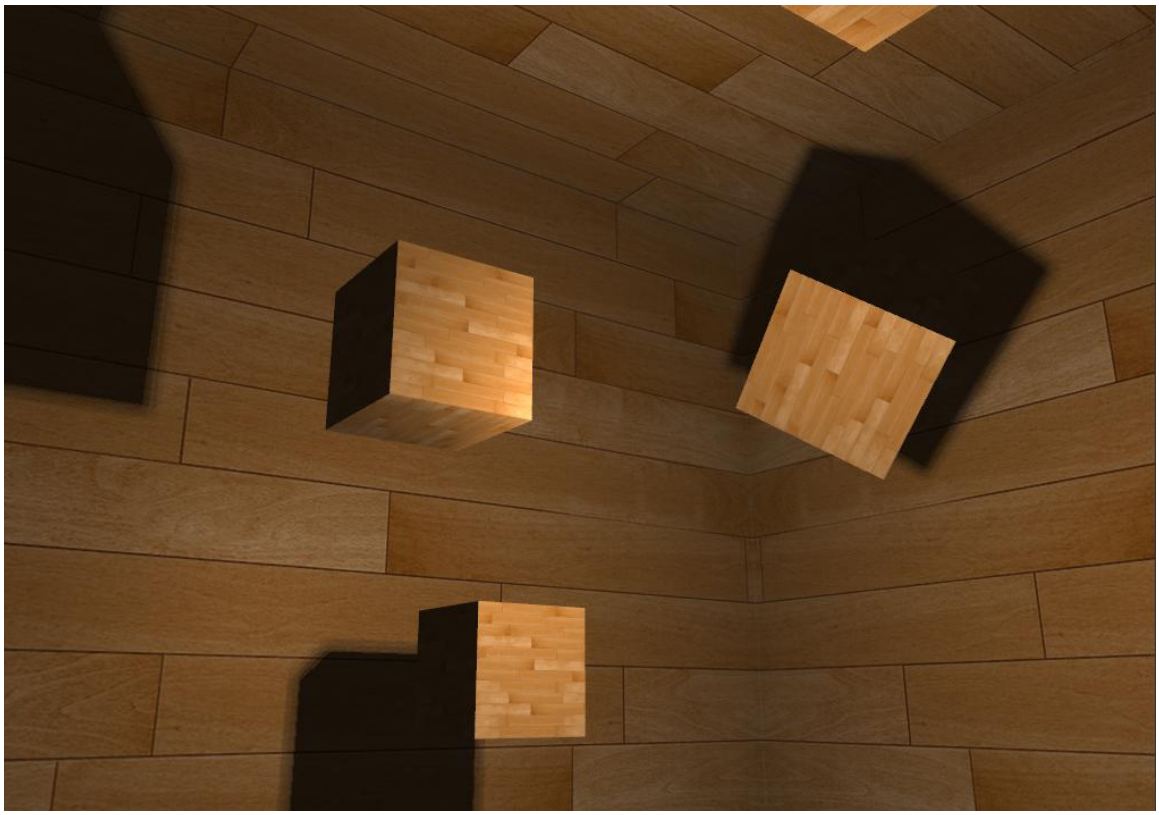
```
vec2 ComputeParallaxMappingTexCoords(vec2 texCoords, vec3 viewDir)
{
    // TODO : make those constants into a uniform buffer
    const float minLayers = 8.0;
    const float maxLayers = 32.0;
    const float heightScale = 0.1;

    // calculate the size of each layer
    const float numLayers = mix(maxLayers, minLayers, abs(dot(vec3(0.0, 0.0, 1.0), viewDir)));
    const float layerDepth = 1.0 / numLayers;

    // depth of current layer
    float currentLayerDepth = 0.0;
    // the amount to shift the texture coordinates per layer (from vector P)
    vec2 parallaxVector = viewDir.xy / viewDir.z * 0.1;
    vec2 deltaTexCoords = parallaxVector / numLayers;

    // get initial values
    vec2  currentTexCoords     = texCoords;
    float currentDepthMapValue = texture(heightMap, currentTexCoords).r;

    while (currentLayerDepth < currentDepthMapValue)
    {
        // shift texture coordinates along direction of parallaxVector
        currentTexCoords -= deltaTexCoords;
        // get height map value at current texture coordinates
        currentDepthMapValue = texture(heightMap, currentTexCoords).r;
        // get depth of next layer
        currentLayerDepth += layerDepth;
    }

    // For parallax occlusion mapping : interpolate between the layer we stopped on and the layer before

    // get texture coordinates before collision (reverse operations)
    vec2 prevTexCoords = currentTexCoords + deltaTexCoords;

    // get depth after and before collision for linear interpolation
```

## Omnidirectional Shadow Mapping

This uses a depth cubemap to generate shadows cast by a point light in all directions.

The lighting fragment shader samples the cubemap with a direction vector to obtain the closest depth (from the light's perspective) at that fragment.

This technique uses PCF (percentage-close filtering) for softer shadows and a shadow bias to prevent shadow acne.



Shader excerpt (fragment shader):

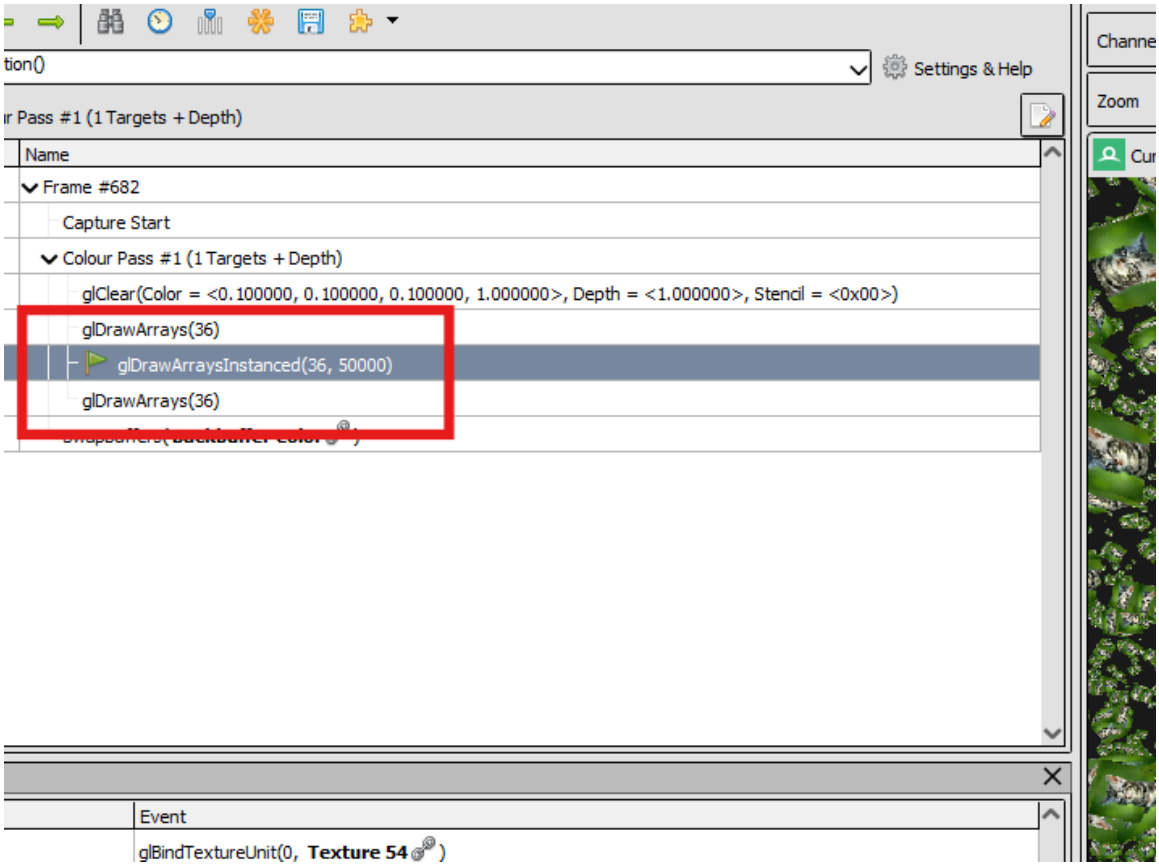
```
float ShadowCalculation(vec4 fragPos, vec4 lightPos)
{
    // In world space
    vec3 lightToFrag = fragPos.xyz - lightPos.xyz;

    // Next we compute the linear depth of the current fragment from the light POV,
    // which is just the length of the vector going there.
    float currentDepth = length(lightToFrag);
    // now test for shadows
    // New shiny disk method

    float shadow = 0.0;
    float bias = 0.15;
    int samples = 20;
    float viewDistance = length(cameraPos_world - fragPos);
    // tweak the radius based on the distance of the viewer to the fragment,
    // making the shadows softer when far away and sharper when close by.
    float diskRadius = (1.0 + (viewDistance / projectionFarPlane)) / 25.0;

    for(int i = 0; i < samples; ++i)
    {
        float closestDepth = texture(depthCubeMap, lightToFrag + gridSamplingDisk[i] * diskRadius).r;
        closestDepth *= projectionFarPlane;   // undo mapping [0;1]

        if (currentDepth - bias > closestDepth)
            shadow += 1.0;
    }

    shadow /= float(samples);

    return shadow;
}
```

## Instancing

A demonstration of instanced drawing, one of the foundations of high-performance rendering. Per-instance data is passed to the shader so that thousands of objects can be drawn in a single draw call.

« It’s better to draw 50,000 kitties once than drawing 1 kitty 50,000 times. »

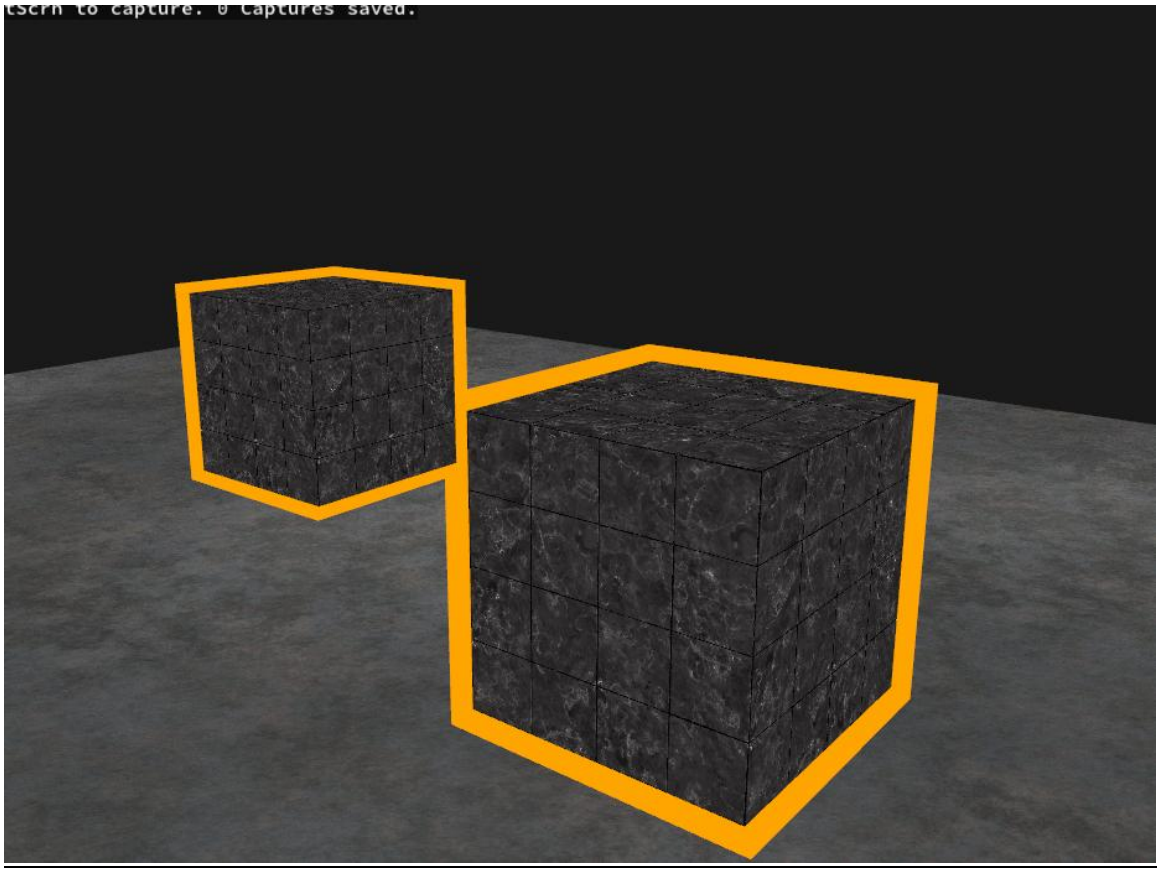
Capture in RenderDoc:



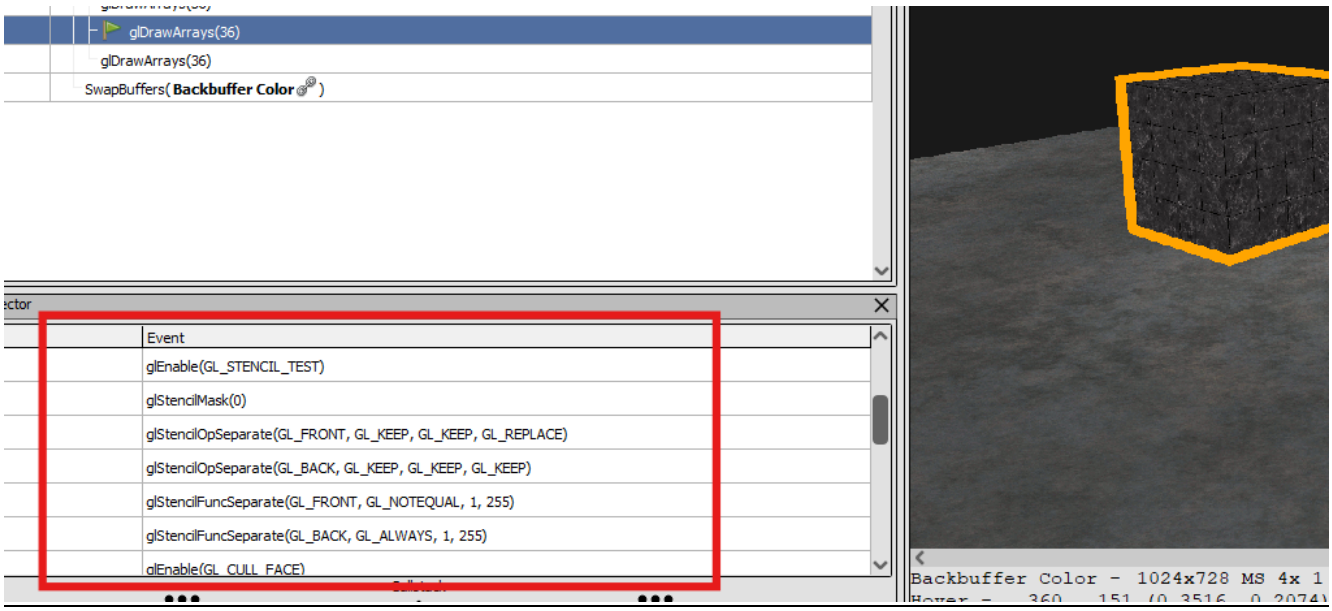
## Stencil Outlines

An implementation of the selection outline effect — commonly used in game editors to highlight selected objects — using the stencil buffer.

Each object is drawn twice with stencil writing enabled: first normally, then scaled up slightly with a solid orange material. The second pass is masked to the stencil boundary, producing a clean silhouette.



# Capture in Renderdoc:

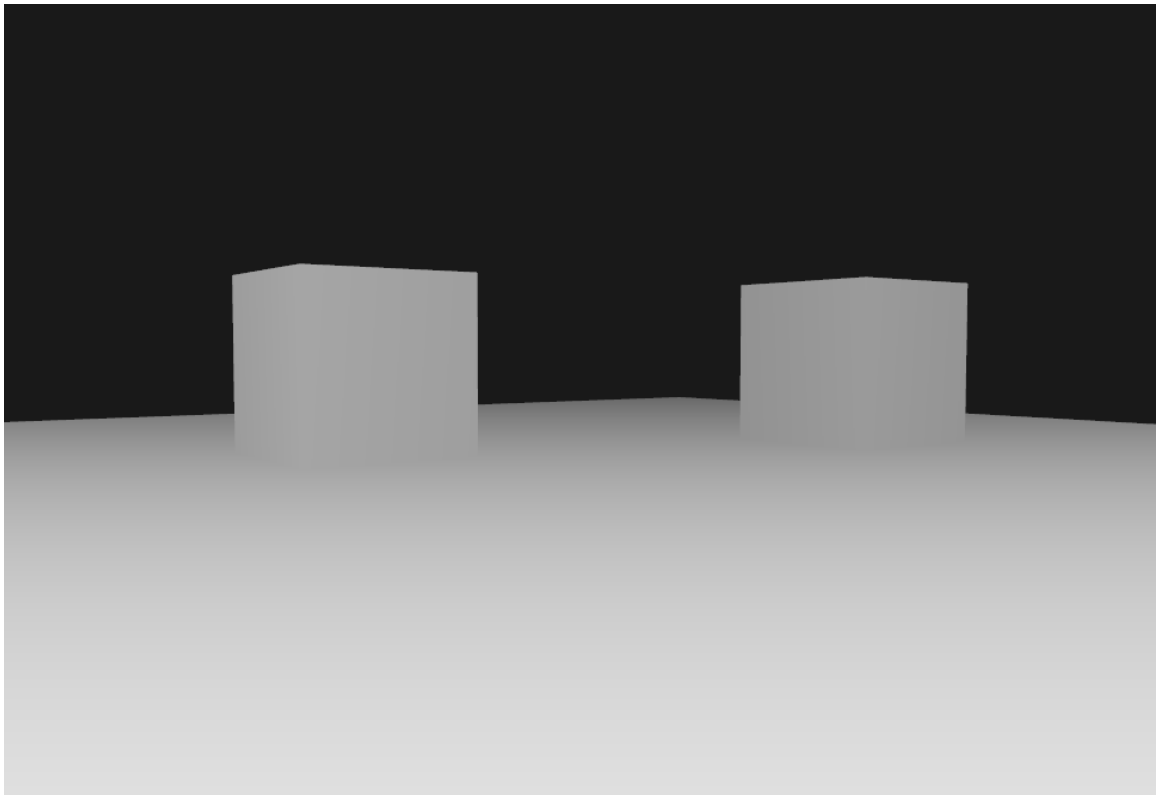


## Depth Buffer Visualizer

A utility shader for visualizing the contents of a depth buffer.

Depth values must be linearized from their non-linear NDC Z values before they can be displayed meaningfully.

In this version, close surfaces appear white. The further an object is, the darker it becomes.



Shader code excerpt (fragment shader):

```
float LinearizeDepth(float depth)
{
    // we have to first re-transform the depth values from the range [0,1] to normalized device coordinates in the range [-1,1].
    // Then we want to reverse the non-linear equation of the projection matrix and apply this inversed equation to the resulting depth value.
    // The result is then a linear depth value.
    float z = depth * 2.0 - 1.0; // back to NDC
    return (2.0 * near * far) / (far + near - z * (far - near));
}


void main()
{
    float depth = LinearizeDepth(gl_FragCoord.z) / far; // divide by far for demonstration
    FragColor = vec4(vec3(depth), 1.0);

    //FragColor = texture(diffuseMap, TexCoords);
}
```